

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ «МИСИС»**

ИНСТИТУТ КОМПЬЮТЕРНЫХ НАУК (ИKN)

КАФЕДРА АВТОМАТИЗИРОВАННЫХ СИСТЕМ УПРАВЛЕНИЯ (АСУ)

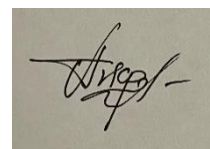
Курс «Клиент-серверные приложения и сетевые
технологии»

Курсовая работа на тему:

**«Разработка web-приложения для управления сотрудниками
компании»**

Выполнил: студент группы БИВТ-24-4

Алфёров Сергей



Проверили:

Абросимов Никита Андреевич

Шелудяков Пётр Алексеевич

Москва, 2026

Содержание

1	Предметная область	4
1.1	Общая характеристика.....	4
1.2	Проблематика	4
2	Постановка задачи	4
2.1	Цель работы	4
2.2	Функциональные требования.....	4
2.3	Требования к программной реализации.....	5
3	Описание архитектуры	5
3.1	Общая архитектура приложения.....	5
3.2	Используемые технологии	5
3.3	Основные сущности системы.....	6
4	Описание структуры базы данных	6
4.1	Вербальная модель	6
4.2	Связи между сущностями.....	7
4.3	Структура массива пользователей.....	Error! Bookmark not defined.
5	Описание серверной части	12
5.1	Структура серверной части	12
5.2	Структура файлов серверной части.....	12
5.3	Диаграмма классов серверной части	13
5.4	Описание ключевых файлов	14
5.5	Примеры API-запросов и ответов.....	16
5.6	Логирование событий	17
5.7	HTTP-коды ответов API	18
5.8	Обработка ошибок и исключений	18
6	Описание клиентской части	19
6.1	Общая структура клиентской части	19
6.2	Интерфейс пользователя.....	20
6.3	Избранное (LocalStorage).....	20
6.4	Админ-панель	21
6.5	React-виджеты	22
7	Заключение	25
7.1	Результаты работы	25

7.2	Решённые задачи	25
7.3	Использованные инструменты.....	26
7.4	Перспективы развития	26
8	Список литературы	26

1 Предметная область

1.1 Общая характеристика

Современные компании всё чаще переходят на цифровые инструменты управления персоналом. Это связано с необходимостью централизованного хранения данных о сотрудниках, контроля доступа к информации и автоматизации рутинных задач.

1.2 Проблематика

На основе анализа существующих решений были выявлены следующие проблемы:

1. отсутствие удобного веб-интерфейса для просмотра списка сотрудников;
2. невозможность сохранять избранных сотрудников в личном кабинете;
3. неудобная система авторизации и разграничения прав доступа;
4. отсутствие встроенного логирования действий пользователей;
5. нет виджетов с топом сотрудников и статистикой.

2 Постановка задачи

Вариант №2

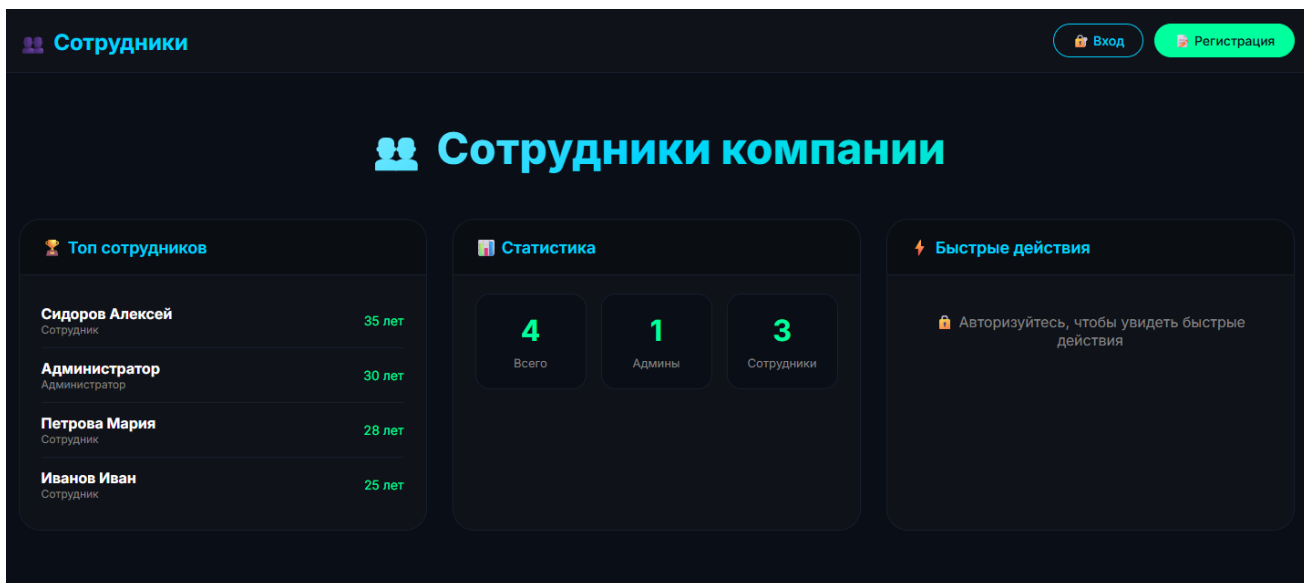
2.1 Цель работы

Разработать полнофункциональное клиент-серверное веб-приложение для управления сотрудниками компании, реализующее авторизацию, разграничение прав, просмотр профилей, систему избранного и панель администратора.

2.2 Функциональные требования

В ходе выполнения курсовой работы необходимо реализовать следующие функции:

1. регистрация и авторизация пользователей с проверкой введённых данных;
2. просмотр списка всех сотрудников;
3. добавление сотрудников в «Избранное» с сохранением в локальном хранилище;
4. личный кабинет с отображением профиля и избранных сотрудников;
5. панель администратора для просмотра всех пользователей и добавления новых;
6. виджеты: топ сотрудников по возрасту, статистика, быстрые действия;
7. журнал событий авторизации и действий пользователей.



Скриншот 1 - Главная страница с виджетами.

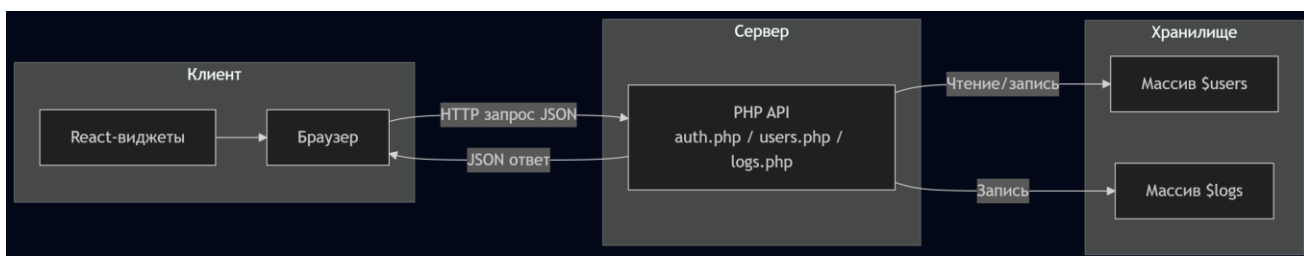
2.3 Требования к программной реализации

Приложение должно быть построено по клиент-серверной архитектуре. Клиентская часть реализуется с использованием HTML, CSS и JavaScript с React-виджетами. Серверная часть разрабатывается на языке PHP. Для хранения данных используется реляционная база данных MySQL. Взаимодействие между клиентом и сервером осуществляется посредством HTTP-запросов с обменом данными в формате JSON.

3 Описание архитектуры

3.1 Общая архитектура приложения

Разработанное приложение построено по классической клиент-серверной архитектуре. Такое решение обеспечивает чёткое разделение ответственности между компонентами системы и упрощает дальнейшее развитие проекта.



Скриншот 2 - Схема архитектуры

3.2 Используемые технологии

При разработке использовались следующие технологии и инструменты:

1. HTML - для создания структуры веб-страниц;
2. CSS - для стилизации интерфейса и обеспечения адаптивности;
3. JavaScript - для реализации клиентской логики;

4. React - для создания интерактивных виджетов;
5. PHP - для разработки серверной части (REST API);
6. MySQL - для хранения данных
7. Apache - в качестве веб-сервера;
8. Git - для контроля версий.

3.3 Основные сущности системы

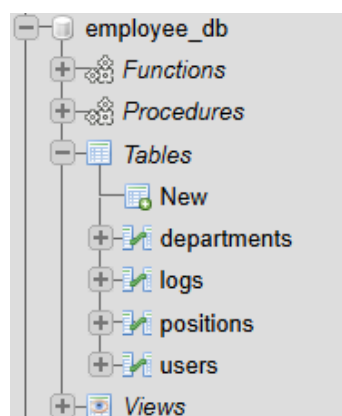
1. Пользователь - сотрудник или администратор, имеющий доступ к системе.
2. Администратор - привилегированный пользователь, имеющий доступ к панели управления, просмотру списка всех пользователей и журнала событий.
3. Гость - неавторизованный посетитель, который может только просматривать главную страницу без возможности управления.
4. Избранное - список идентификаторов сотрудников, сохранённый в локальном хранилище браузера.

4 Описание структуры базы данных

4.1 Вербальная модель

База данных `employee_db` содержит следующие сущности:

1. Пользователи (`users`) - хранит учётные записи всех зарегистрированных пользователей. Каждая запись содержит уникальный идентификатор, имя, email, возраст, логин, хэш пароля, роль (`admin/employee`), внешние ключи на отдел и должность, дату регистрации.
2. Логи (`logs`) - хранит историю событий авторизации и действий. Фиксирует дату и время события, имя пользователя, тип действия, IP-адрес и детали.
3. Отделы (`departments`) - хранит информацию об отделах компании. Содержит название отдела и описание.
4. Должности (`positions`) - хранит информацию о должностях. Содержит название должности и уровень.



Скриншот 3 - Структура таблиц базы данных в phpMyAdmin.

4.2 Связи между сущностями

Типы связей:

- users -> logs - один ко многим (один пользователь может иметь много записей в логах)
- users -> departments - многие к одному (много пользователей могут принадлежать одному отделу)
- users -> positions - многие к одному (много пользователей могут иметь одну должность)

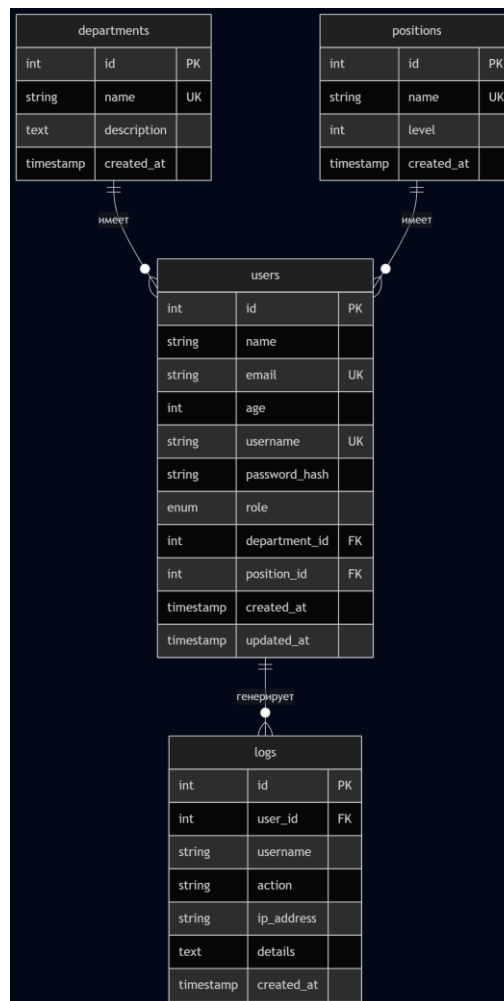


Рисунок 2 - Структура базы данных

4.3 SQL-код создания таблиц

sql

```
CREATE TABLE users (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100) NOT NULL,  
    email VARCHAR(100) UNIQUE NOT NULL,  
    age INT,  
    username VARCHAR(50) UNIQUE NOT NULL,  
    password_hash VARCHAR(255) NOT NULL,
```

```

        role ENUM('admin', 'employee') DEFAULT 'employee',
        department_id INT NULL,
        position_id INT NULL,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    );

CREATE TABLE logs (
    id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT NULL,
    username VARCHAR(50) NOT NULL,
    action VARCHAR(50) NOT NULL,
    ip_address VARCHAR(45) NOT NULL,
    details TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET NULL
);

CREATE TABLE departments (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL UNIQUE,
    description TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE positions (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL UNIQUE,
    level INT DEFAULT 1,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

Листинг 1 - Создание таблиц

sql

```

CREATE VIEW v_active_users AS
SELECT id, name, email, username, role, created_at
FROM users WHERE role = 'employee';

```



```

CREATE VIEW v_users_with_details AS
SELECT u.id, u.name, u.email, u.age, u.username, u.role,
       d.name AS department, p.name AS position, p.level AS
position_level
FROM users u
LEFT JOIN departments d ON u.department_id = d.id
LEFT JOIN positions p ON u.position_id = p.id;

CREATE VIEW v_recent_logs AS
SELECT id, username, action, ip_address, details,
       DATE(created_at) AS log_date, TIME(created_at) AS log_time
FROM logs
WHERE created_at >= DATE_SUB(NOW(), INTERVAL 7 DAY)
ORDER BY created_at DESC;

```

Листинг 2 - Представления

sql

```

CREATE FUNCTION get_user_count_by_role(p_role VARCHAR(20))
RETURNS INT DETERMINISTIC
BEGIN
    DECLARE user_count INT;
    SELECT COUNT(*) INTO user_count FROM users WHERE role =
p_role;
    RETURN user_count;
END;

CREATE FUNCTION get_username_by_id(p_id INT)
RETURNS VARCHAR(100) DETERMINISTIC
BEGIN
    DECLARE user_name VARCHAR(100);
    SELECT name INTO user_name FROM users WHERE id = p_id;
    RETURN user_name;
END;

CREATE FUNCTION get_log_count_by_action(p_action VARCHAR(50))
RETURNS INT DETERMINISTIC
BEGIN

```

```

        DECLARE log_count INT;

        SELECT COUNT(*) INTO log_count FROM logs WHERE action =
p_action;

        RETURN log_count;

    END;

```

Листинг 3 - Функции

sql

```

DELIMITER //

CREATE PROCEDURE sp_add_user(
    IN p_name VARCHAR(100),
    IN p_email VARCHAR(100),
    IN p_age INT,
    IN p_username VARCHAR(50),
    IN p_password VARCHAR(255),
    IN p_role VARCHAR(20),
    IN p_department_id INT,
    IN p_position_id INT
)
BEGIN
    INSERT INTO users (name, email, age, username, password_hash,
role, department_id, position_id)
        VALUES (p_name, p_email, p_age, p_username, p_password,
p_role, p_department_id, p_position_id);
    END //

CREATE PROCEDURE sp_delete_user(IN p_id INT)
BEGIN
    DELETE FROM users WHERE id = p_id;
    END //

CREATE PROCEDURE sp_clear_old_logs(IN p_days INT)
BEGIN
    DELETE FROM logs WHERE created_at < DATE_SUB(NOW(), INTERVAL
p_days DAY);
    END //

```

```
DELIMITER ;
```

Листинг 4 - Хранимые процедуры

sql

```
DELIMITER //

CREATE TRIGGER trg_user_insert
AFTER INSERT ON users
FOR EACH ROW
BEGIN
    INSERT INTO logs (user_id, username, action, ip_address,
details)
    VALUES (NEW.id, NEW.username, 'USER_INSERT', '127.0.0.1',
CONCAT('Добавлен пользователь: ', NEW.name));
END //

CREATE TRIGGER trg_user_update
AFTER UPDATE ON users
FOR EACH ROW
BEGIN
    IF OLD.name != NEW.name OR OLD.email != NEW.email OR OLD.role
!= NEW.role THEN
        INSERT INTO logs (user_id, username, action, ip_address,
details)
        VALUES (NEW.id, NEW.username, 'USER_UPDATE', '127.0.0.1',
CONCAT('Обновлён пользователь: ', OLD.name, ' → ', NEW.name));
    END IF;
END //

CREATE TRIGGER trg_user_delete
BEFORE DELETE ON users
FOR EACH ROW
BEGIN
    INSERT INTO logs (user_id, username, action, ip_address,
details)
    VALUES (OLD.id, OLD.username, 'USER_DELETE', '127.0.0.1',
```

```
CONCAT('Удалён пользователь: ', OLD.name));  
END //
```



```
DELIMITER ;
```

Листинг 5 - Триггеры

5 Описание серверной части

5.1 Структура серверной части

Серверная часть приложения разработана на языке PHP и организована по принципу разделения ответственности. Выделены следующие компоненты:

1. **API-маршрутизатор** - обрабатывает входящие HTTP-запросы и возвращает данные в формате JSON;
2. **Обработчики форм** - обрабатывают данные, отправленные из HTML-форм (регистрация, авторизация);
3. **Модели данных** - функции для работы с хранилищем (получение списков, CRUD операции);
4. **Логирование** - запись событий авторизации в массив и вывод в админ-панели.

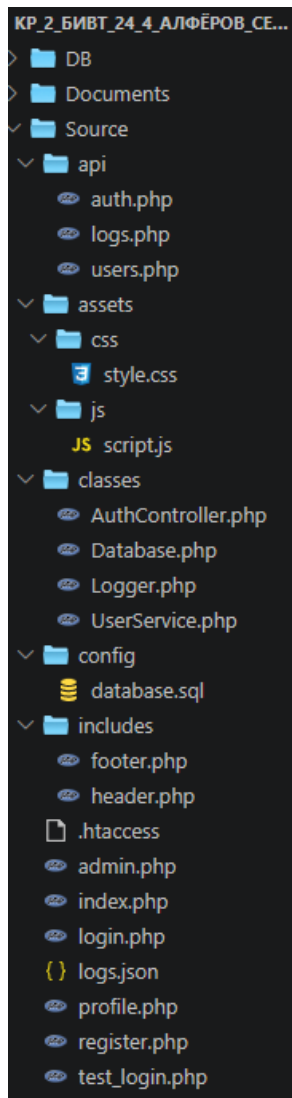
5.2 Структура файлов серверной части

Рисунок 3 - Структура файлов

Source/

```
|— api/  
|   |— auth.php      # аутентификация  
|   |— users.php     # CRUD пользователей  
|   |— logs.php      # получение логов  
|— classes/  
|   |— Database.php  # работа с MySQL  
|   |— Logger.php    # логирование  
|   |— AuthController.php  
|   |— UserService.php  
|— includes/  
|   |— header.php  
|   |— footer.php  
|— index.php        # главная страница  
|— login.php  
|— register.php  
|— profile.php
```

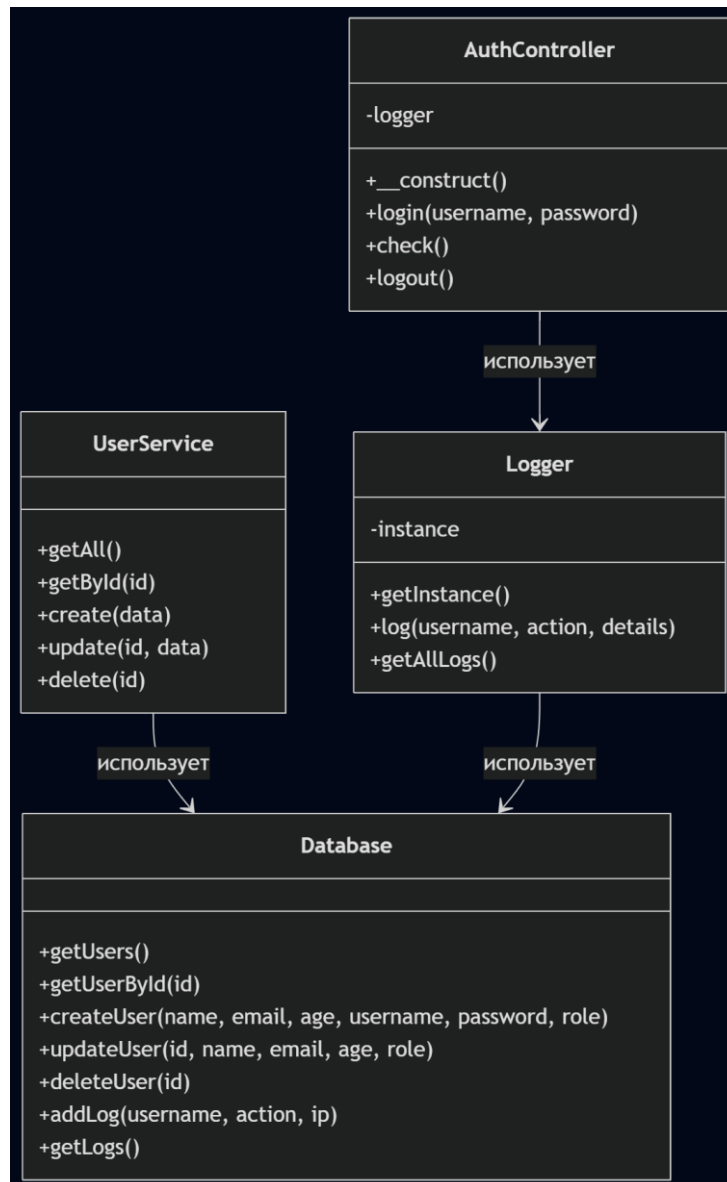
```
├── admin.php
├── assets/
│   ├── css/style.css
│   └── js/script.js
```



Скриншот 4 - Дерево папок проекта в VSCode.

5.3 Диаграмма классов серверной части

Рисунок 4 - Диаграмма классов



Скриншот 5 - Диаграмма классов (AuthController, UserService, Logger, Database).

5.4 Описание ключевых файлов

Файл `classes/Database.php` содержит массив пользователей, функции для получения списков, создания, обновления и удаления пользователей, а также массив логов.

php

```

class Database {
    public static function getConnection() { ... }
    public static function getAllUsers() { ... }
    public static function getUserById($id) { ... }
    public static function createUser($data) { ... }
    public static function updateUser($id, $data) { ... }
    public static function deleteUser($id) { ... }
    public static function getAllLogs() { ... }
}
  
```

```

        public static function addLog($username, $action, $ip,
$details = null) { ... }
        public static function getStats() { ... }
        public static function getTopEmployees() { ... }
        public static function findUserByUsername($username) { ... }
    }

```

Листинг 3 - Основные методы класса Database

Файл api/auth.php - обрабатывает запросы на вход, проверку сессии и выход.

php

```

session_start();
header('Content-Type: application/json');
require_once '../classes/Database.php';
require_once '../classes/Logger.php';
require_once '../classes/AuthController.php';

$auth = new AuthController();

if ($_SERVER['REQUEST_METHOD'] === 'GET' &&
isset($_GET['check'])) {
    echo json_encode($auth->check());
    exit;
}

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $data = json_decode(file_get_contents('php://input'), true);
    $result = $auth->login($data['username'] ?? '',
$data['password'] ?? '');
    echo json_encode($result);
    exit;
}

```

Листинг 4 - API аутентификации

Файл admin.php - панель администратора. Доступна только пользователям с ролью admin.

php

```
session_start();  
if (!isset($_SESSION['role']) || $_SESSION['role'] !== 'admin') {  
    header('Location: login.php');  
    exit;  
}
```

Листинг 5 - Проверка прав администратора

5.5 Примеры API-запросов и ответов

GET /Source/api/users.php

GET - запрос не имеет тела

Листинг 6 - Запрос на получение списка пользователей

json

```
{  
    "status": "success",  
    "data": [  
        {  
            "id": 1,  
            "name": "Администратор",  
            "email": "admin@test.com",  
            "role": "admin"  
        },  
        {  
            "id": 2,  
            "name": "Иванов Иван",  
            "email": "ivan@test.com",  
            "role": "employee"  
        }  
    ]  
}
```

Листинг 7 - Ответ сервера


```
[{"id":1,"name":"\u0410\u0434\u043c\u0438\u0434\u0438\u0441\u0442\u0440\u0430\u0442\u0430\u0435\u0440","email":"admin@test.com","age":30,"username":"admin","password":"$2y$10$0yF5xFGt6S086jgPHV3kG0aGLUYwTLXKL8IifNBX8sS1ONMzSKWs6","role":"admin","created_at":"2026-06-05 14:41:47"}, {"id":2,"name":"\u0418\u0432\u0430\u043d\u043e\u0432","email":"ivan@test.com","age":25,"username":"ivanov","password":"$2y$10$nW5n2kWUqb5/CWvMKWJAOfk4DQ9XE4qkIA1kBgW1P1zVL4QyfwDe","role":"employee","created_at":"2026-06-05 14:41:47"}, {"id":3,"name":"\u041c\u0430\u0440\u0438\u044f","email":"maria@test.com","age":28,"username":"petrova","password":"$2y$10$XARl39v7KHcIR5gZFTytB0lc\sr1MwU6vo9SXSrEYKi57D7xGxLxW","role":"employee","created_at":"2026-06-05 14:41:47"}, {"id":4,"name":"\u0410\u043b\u0435\u043a\u0441\u0430\u0441\u043e\u0432","email":"alex@test.com","age":35,"username":"sidorov","password":"$2y$10$/xtOpXeiDzeaZZn7w7TRcOy1/WfQLVFC5ZXTI2aaUxM/QZ75jDBiy","role":"employee","created_at":"2026-06-05 14:41:47"}]
```

Скриншот 6 - Пример ответа API в браузере или Postman.

POST /Source/api/auth.php - авторизация

Запрос:

json

```
{
  "username": "admin",
  "password": "admin123"
}
```

Ответ при успехе:

json

```
{
  "success": true,
  "role": "admin"
}
```

5.6 Логирование событий

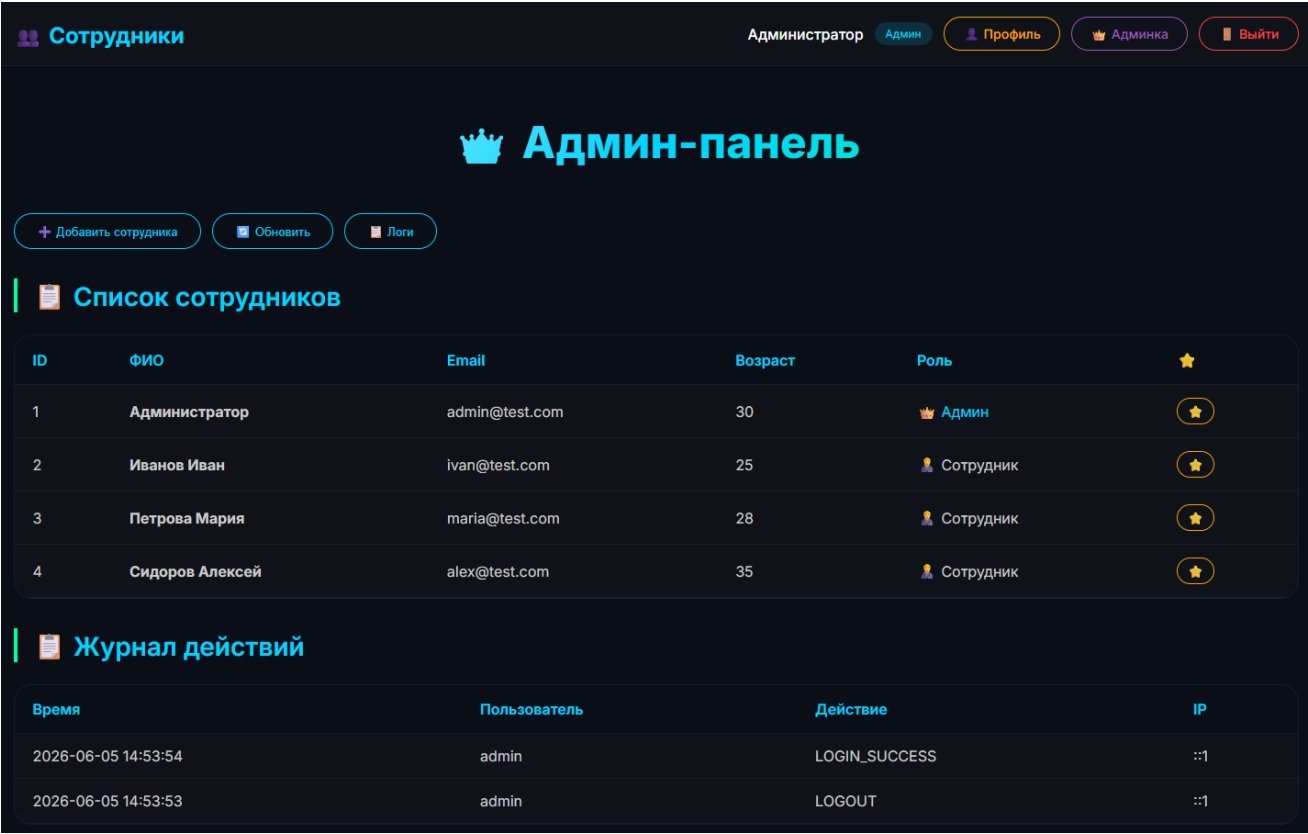
Все события авторизации и действий записываются:

1. В массив \$logs в памяти
2. Выводятся в админ-панели через API /api/logs.php

Логируемые события:

- LOGIN_SUCCESS - успешный вход
- LOGIN_FAILED - неудачная попытка входа

- LOGOUT - выход из системы
- CREATE_USER - создание пользователя (админ)
- UPDATE_USER - обновление пользователя
- DELETE_USER - удаление пользователя



Скриншот 7 - Таблица логов в админ-панели.

5.7 HTTP-коды ответов API

HTTP-код	Значение	Ситуация использования
200	OK	Успешный GET/POST-запрос
400	Bad Request	Неверные параметры запроса
401	Unauthorized	Пользователь не авторизован
403	Forbidden	Недостаточно прав доступа
404	Not Found	Пользователь не найден
405	Method Not Allowed	Неподдерживаемый метод
500	Internal Server Error	Ошибка на стороне сервера

Таблица 1 - HTTP-коды

5.8 Обработка ошибок и исключений

php

```
try {
```

```

    $users = getUsers();
    echo json_encode(['status' => 'success', 'data' => $users]);
} catch (Exception $e) {
    http_response_code(500);
    echo json_encode(['status' => 'error', 'message' => 'Ошибка
сервера']);
}

```

Листинг 8 - Обработка ошибок в API

Типы обрабатываемых ошибок:

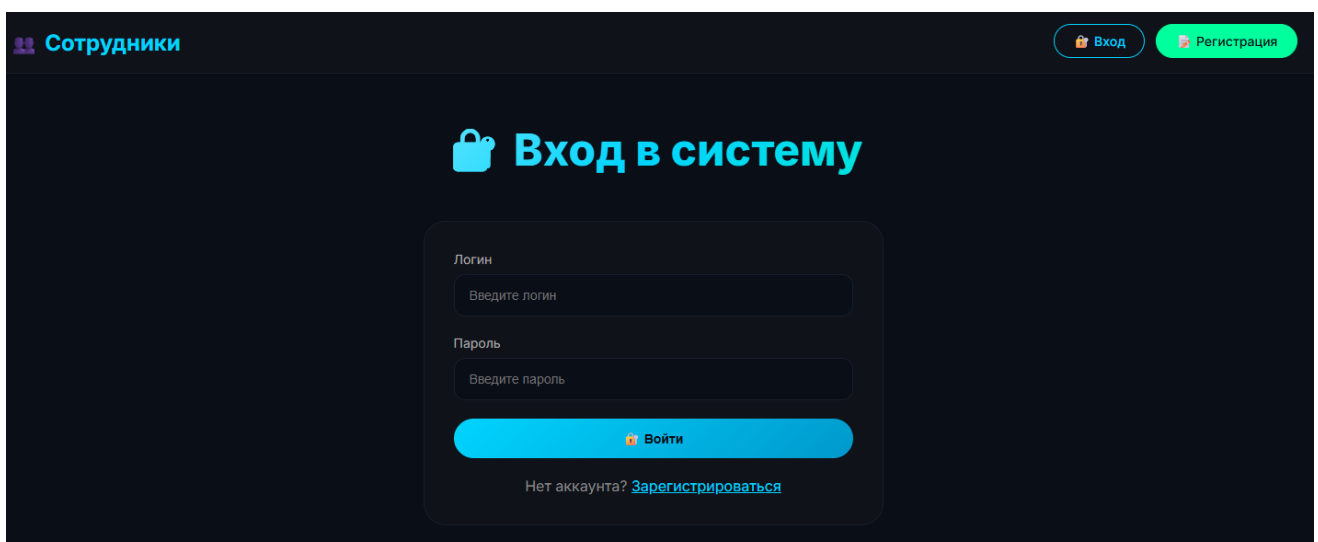
1. Ошибки валидации данных (пустые поля, неверный email)
2. Ошибки авторизации (неверный логин/пароль)
3. Ошибки прав доступа (403 Forbidden)
4. Ресурс не найден (404 Not Found)

6 Описание клиентской части

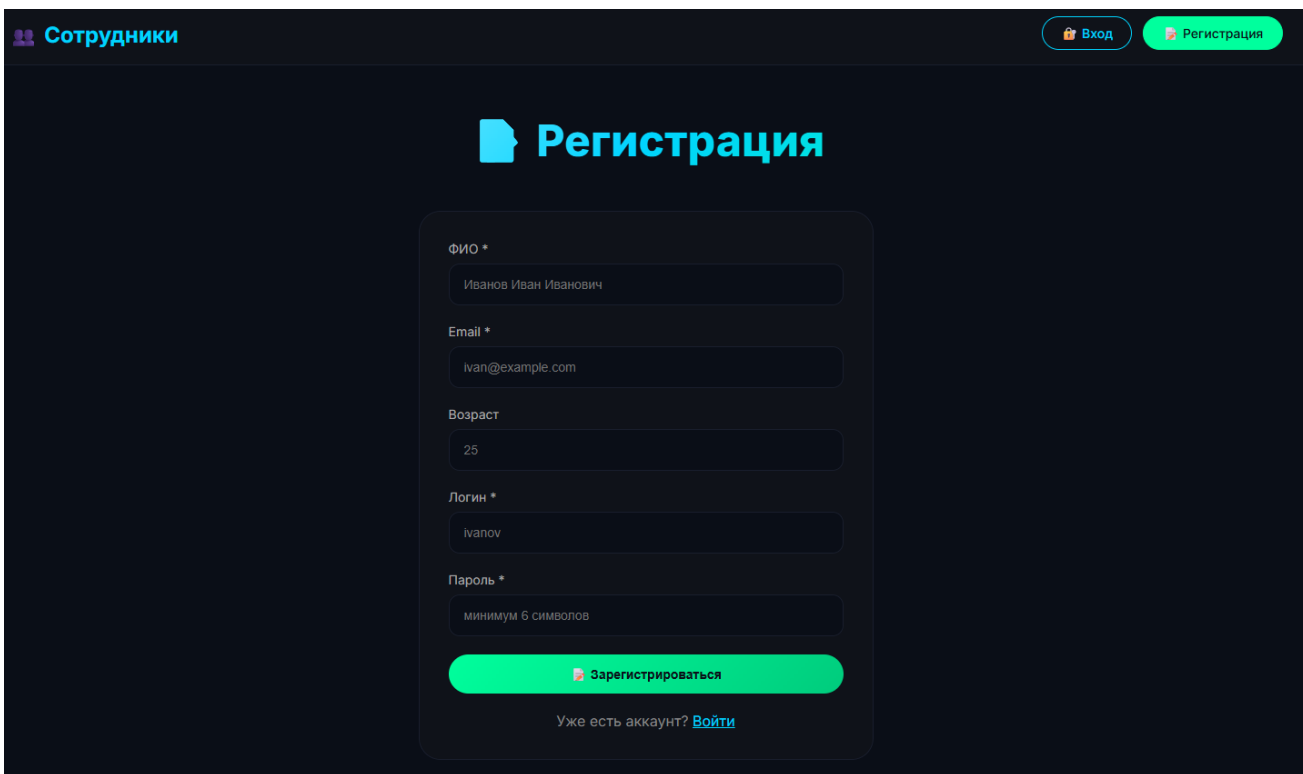
6.1 Общая структура клиентской части

Клиентская часть приложения включает следующие страницы и компоненты:

1. **Главная страница** - содержит React-виджеты с топом сотрудников, статистикой и быстрыми действиями;
2. **Страница входа** - форма для авторизации;
3. **Страница регистрации** - создание новой учётной записи;
4. **Личный кабинет** - отображает профиль и избранных сотрудников;
5. **Админ-панель** - доступна только администраторам (список пользователей, добавление, логи).



Скриншот 8 - Страница входа.



Скриншот 9 - Страница регистрации.

6.2 Интерфейс пользователя

Главная страница содержит три React-виджета:

1. **Топ сотрудников** - 5 самых возрастных сотрудников
2. **Статистика** - общее количество, администраторов, сотрудников
3. **Быстрые действия** - кнопки перехода в профиль, админку, выход

6.3 Избранное (LocalStorage)

Реализация сохранения избранных сотрудников в локальном хранилище браузера:

javascript

```
function saveToLocalStorage(key, data) {  
    localStorage.setItem(key, JSON.stringify(data));  
}  
  
function getFromLocalStorage(key) {  
    const data = localStorage.getItem(key);  
    return data ? JSON.parse(data) : null;  
}  
  
function addToFavorites(employeeId) {
```

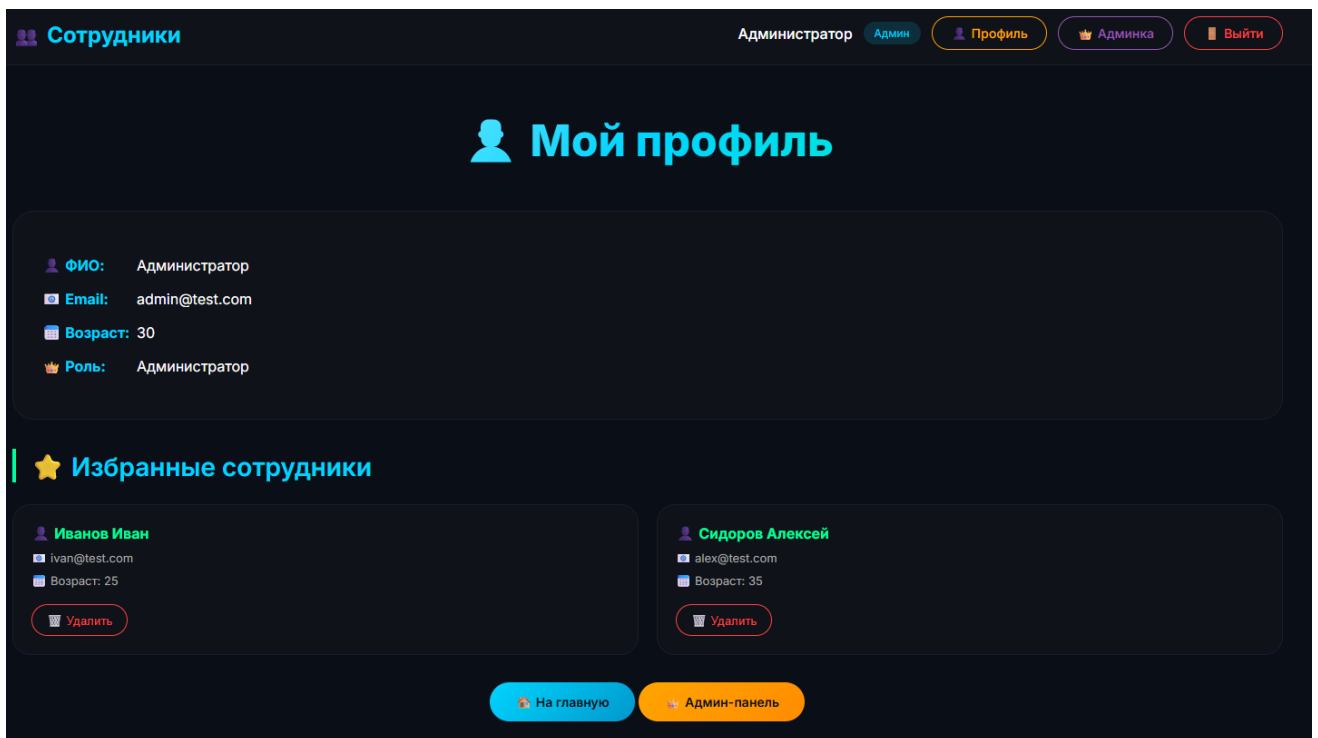
```

    let favorites = getFromLocalStorage('favorite_employees') ||
[];

    if (!favorites.includes(employeeId)) {
        favorites.push(employeeId);
        saveToLocalStorage('favorite_employees', favorites);
        alert('✓ Добавлен в избранное');
    }
}

```

Листинг 9 - Функции сохранения и загрузки избранного



Скриншот 10 - Личный кабинет с избранными сотрудниками.

6.4 Админ-панель

Админ-панель реализует следующие возможности:

- Просмотр всех зарегистрированных пользователей
- Добавление новых сотрудников
- Просмотр журнала действий (логов)

javascript

```

async function addUser() {
    const data = {
        name: document.getElementById('newName').value,
        email: document.getElementById('newEmail').value,
        age: document.getElementById('newAge').value,

```

```

        username: document.getElementById('newUsername').value,
        password: document.getElementById('newPassword').value,
        role: document.getElementById('newRole').value
    };

    const res = await fetch('/Source/api/users.php', {
        method: 'POST',
        headers: {'Content-Type': 'application/json'},
        body: JSON.stringify(data)
    });
    // ...
}

```

Листинг 10 - Добавление пользователя (админ-панель)

Список сотрудников					
ID	ФИО	Email	Возраст	Роль	★
1	Администратор	admin@test.com	30	👑 Админ	★
2	Иванов Иван	ivan@test.com	25	👤 Сотрудник	★
3	Петрова Мария	maria@test.com	28	👤 Сотрудник	★
4	Сидоров Алексей	alex@test.com	35	👤 Сотрудник	★

Скриншот 11 - Админ-панель (список пользователей).

Журнал действий			
Время	Пользователь	Действие	IP
2026-06-05 14:56:42	admin	LOGIN_SUCCESS	::1
2026-06-05 14:54:44	admin	LOGOUT	::1
2026-06-05 14:53:54	admin	LOGIN_SUCCESS	::1
2026-06-05 14:53:53	admin	LOGOUT	::1

Скриншот 12 - Журнал действий (логи).

6.5 React-виджеты

На главной странице реализованы три React-компонента (подключены через CDN с Babel):

jsx

```

const TopWidget = () => {
    if (topEmployees.length === 0) {
        return <div style={{ textAlign: 'center', color: '#888',
padding: '30px' }}>Нет данных</div>;
    }
}

```

```

    }

    return (
      <div>
        {topEmployees.map((emp, idx) => (
          <div key={emp.id} className="employee-item">
            <div>
              <div className="employee-
name"><strong>{emp.name}</strong></div>
              <div style={{ fontSize: '12px', color:
'#888' }}>
                {emp.role === 'admin' ? '👑
Администратор' : '👤 Сотрудник'}
              </div>
            </div>
          <div className="employee-age">{emp.age ||
'?'} лет</div>
        </div>
        )))
      </div>
    );
  };
};

```

Листинг 11 - React-виджет "Топ сотрудников"

jsx

```

const StatsWidget = () => {
  return (
    <div className="stats-row">
      <div className="stat-box">
        <div className="stat-
number">{statsData.total}</div>
        <div className="stat-label">Всего</div>
      </div>
      <div className="stat-box">
        <div className="stat-
number">{statsData.admins}</div>

```

```

        <div className="stat-label">Админы</div>
      </div>
      <div className="stat-box">
        <div className="stat-
number">{statsData.employees}</div>
        <div className="stat-label">Сотрудники</div>
      </div>
    </div>
  );
};

```

Листинг 12 - React-виджет "Статистика"

jsx

```

const ActionsWidget = () => {
  if (!isLoggedIn) {
    return <div>🔒 Авторизуйтесь, чтобы увидеть быстрые
действия</div>;
  }
  return (
    <div className="actions-list">
      <a href="profile.php" className="action-btn action-
btn-primary">👤 Мой профиль</a>
      {isAdmin && <a href="admin.php" className="action-btn
action-btn-warning">👑 Админ-панель</a>}
      <a href="api/auth.php?logout=1" className="action-btn
action-btn-danger">🚪 Выйти</a>
    </div>
  );
};

```

Листинг 13 - React-виджет "Быстрые действия"

Используемые React-хуки:

- useState - для управления состоянием
- useEffect - для загрузки данных при монтировании


```

const TopWidget = () => {
  if (topEmployees.length === 0) {
    return <div style={{ textAlign: 'center', color: '#888', padding: '30px' }}> Нет данных</div>;
  }

  return (
    <div>
      {topEmployees.map((emp, idx) => (
        <div key={emp.id} className="employee-item">
          <div>
            <div className="employee-name"><strong>{emp.name}</strong></div>
            <div style={{ fontSize: '12px', color: '#888' }}>{emp.role === 'admin' ? 'Администратор' : 'Сотрудник'}</div>
          </div>
          <div className="employee-age">{emp.age || '?'} лет</div>
        </div>
      ))}
    </div>
  );
};

const StatsWidget = () => {
  return (
    <div className="stats-row">
      <div className="stat-box">
        <div className="stat-number">{statsData.total}</div>
        <div className="stat-label">Всего</div>
      </div>
      <div className="stat-box">
        <div className="stat-number">{statsData.admins}</div>
        <div className="stat-label">Админы</div>
      </div>
      <div className="stat-box">
        <div className="stat-number">{statsData.employees}</div>
        <div className="stat-label">Сотрудники</div>
      </div>
    </div>
  );
};

```

Скриншот 13 - Код React-виджета "Статистика"

```

const ActionsWidget = () => {
  if (!isLoggedIn) {
    return (
      <div className="actions-list">
        <div style={{ textAlign: 'center', padding: '20px', color: '#888' }}> Авторизуйтесь, чтобы увидеть быстрые действия</div>
      </div>
    );
  }

  return (
    <div className="actions-list">
      <div style={{ textAlign: 'center', marginBottom: '10px', padding: '8px', background: 'rgba(0,212,255,0.1)', borderRadius: '12px' }}>
        Привет, <strong>{userName}</strong>
      </div>
      <a href="profile.php" className="action-btn action-btn-primary"> Мой профиль</a>
      {isAdmin && <a href="admin.php" className="action-btn action-btn-warning"> Админ-панель</a>}
      <a href="api/auth.php?logout=1" className="action-btn action-btn-danger"> Выйти</a>
    </div>
  );
};

```

Скриншот 14 - Код React-виджета "Быстрые действия"

7 Заключение

7.1 Результаты работы

В ходе выполнения курсовой работы было разработано клиент-серверное веб-приложение «Сотрудники компании». Проект реализован в соответствии с поставленными требованиями и включает все необходимые функции.

7.2 Решённые задачи

В процессе разработки были решены следующие задачи:

1. проведён анализ предметной области и выявлены основные проблемы;
2. спроектирована структура реляционной базы данных MySQL (4 таблицы, 3 представления, 3 функции, 3 процедуры, 3 триггера)
3. разработана серверная часть на языке PHP с реализацией REST API;

4. создана клиентская часть с использованием HTML, CSS и JavaScript;
5. реализованы интерактивные виджеты на React;
6. обеспечено сохранение избранного в локальном хранилище браузера;
7. внедрена система авторизации с разграничением прав доступа;
8. организовано логирование событий авторизации и действий.

7.3 Используемые инструменты

При разработке использовались: HTML, CSS, JavaScript, React, PHP, Apache.

7.4 Перспективы развития

В дальнейшем проект может быть расширен следующими функциями:

1. подключение реальной базы данных MySQL;
2. редактирование и удаление сотрудников;
3. фильтрация и поиск по списку сотрудников;
4. импорт/экспорт данных (CSV/Excel);
5. REST API с JWT-авторизацией.

8 Список литературы

1. Дронов В.А. HTML и CSS. Современные веб-технологии. - СПб.: БХВ-Петербург, 2021. - 480 с.
2. Флэнаган Д. JavaScript. Подробное руководство. - М.: Вильямс, 2022. - 720 с.
3. Никсон Р. Создаём динамические веб-сайты с помощью PHP, MySQL, JavaScript, CSS и HTML5. - СПб.: Питер, 2021. - 768 с.
4. Бэнкс А., Порселло Е. React и TypeScript. Быстрая разработка современных приложений. - СПб.: Питер, 2022. - 512 с.
5. Кузнецов М.В., Симдянов И.В. PHP 7. Полное руководство. - СПб.: БХВ-Петербург, 2020. - 688 с.
6. Ссылка на гитхаб: <https://github.com/nfgone23/employee-management>